

Turning fragmented Chilean banking data into a structured analytics and research platform.

This project was built to make regulatory banking data easier to trust, consume, compare, and explain. It combines automated data ingestion workers, a structured Supabase data layer, a frontend analytics experience, and a guarded LLM research assistant that adds market context without changing source-of-truth data.

MOTIVATION

Why this project exists

Chilean banking information is published across multiple datasets, with different metric definitions, entity naming, refresh patterns, and operational formats. That creates friction for anyone trying to answer simple strategic questions such as which banks are gaining share, how card activity is evolving, or how balances behave across account segments.

The goal is not only to centralize data, but to transform it into a product where analysts can move from raw reporting to decision-ready signals.

5

Active ETL subsystems covering UF, credit cards, debit cards, prepaid cards, and checking accounts.

USER PAIN POINTS

What users struggle with today

Without a consolidated layer, users spend more time cleaning data than interpreting it.

Fragmented sources

Data arrives from multiple regulatory endpoints and time series. Users must reconcile differing tags, months, institutions, and metric shapes before any analysis can begin.

Low comparability

Comparable views such as market share, growth, activation rates, or UF-adjusted balances are not immediately available. Analysts have to manually derive them, which increases inconsistency and error risk.

Missing context

Even when the numbers are available, users still need to understand what happened in the market. Raw charts do not explain regulatory changes, issuer strategy, or external events that shaped the results.

SOLUTION

The product response

The system addresses the problem in layers: ingestion, normalization, access control, visualization, and assisted research.

1. Automated ingestion

Dedicated workers fetch source datasets on a recurring basis, detect source freshness, and load only new periods instead of reprocessing everything blindly.

2. Curated storage

Raw rows are preserved, while curated tables standardize institution codes, operation types, totals, rates, and UF-adjusted metrics needed for analysis.

3. Frontend analytics

The web app consumes the database through backend API routes, exposing trusted dashboards for credit, debit, prepaid, and checking products.

4. Research assistance

A guarded LLM flow combines dashboard context with web search results to help users interpret market dynamics without altering protected data.

HIGH-LEVEL ARCHITECTURE

How the system works end to end

Three layers carry the product: automated workers, the Supabase database, and the frontend/API consumption layer.

LAYER 1

Automated workers

Python workers run the ETL pipelines for UF, credit-card operations, debit-card operations, prepaid-card operations, and checking accounts.

- Call CMF and related source endpoints
- Track sync state per dataset or endpoint
- Retry on transient failures
- Write both raw and curated outputs



LAYER 2

Database and metrics layer

Supabase stores the operational foundation: source registry, sync state, raw tables, curated tables, views, and security-related tables for application access.

- Raw tables preserve ingested observations
- Curated tables expose normalized business entities
- Views publish metrics ready for dashboards
- RLS and audit foundations support protected access



LAYER 3

Frontend consuming the data

The Next.js frontend does not depend on browser-direct analytics queries by default. It consumes backend API routes, which shape public metrics, protect derived metrics, and keep the client focused on presentation and interaction.

- Dashboards for cards and checking accounts
- Bank selection, date filters, and chart views
- Public and protected API separation
- Auth-aware access to advanced analysis surfaces

Core idea: the workers create reliable structure, the database preserves and secures it, and the frontend turns it into a usable analytical product.

LLM RESEARCH LAYER

What the LLM adds, and what it is allowed to do

The LLM is used as a research and explanation layer, not as the authoritative data engine.

Research motivation

Data dashboards answer *what* changed. The LLM research layer is designed to help answer *why* it changed by combining dashboard context with external market information such as news, strategy announcements, and regulation.

1. Contextual input

The frontend sends the user query plus dashboard context such as product, operation, selected banks, period, and time-series values.

2. Retrieval

A backend search route retrieves Chile-focused web results and augments the search with the active dashboard context.

3. Summarization

An OpenAI model produces a structured answer that separates dashboard facts from web-source claims and cites the retrieved sources.

Research safeguards

The LLM is intentionally constrained. It is authenticated, rate-limited, audited, and blocked from behaving like an agent that can write data or execute actions.

- Read-only scope for financial research
- Prompt-injection pattern blocking
- Clear separation between dashboard data and web sources
- No authority to modify databases, pipelines, or settings
- Audit logging for accepted, rejected, and limited requests

In research terms, this makes the LLM a controlled interpretation layer on top of verified internal data and retrieved external evidence, rather than a free-form decision engine.

Summary: the project combines automated ETL, a governed data platform, a dashboard frontend, and a constrained LLM research layer to reduce manual analysis work and produce faster, more explainable financial insight.